搭积木

题目描述

搭积木

有编号为 0 − (N − 1) 0−(N−1)的若干个积木块按编号从小到大的顺序排成一行，每个积木块所在的位置为其对应的编号。一个机器人可以操纵这些积木块，机器人可以执行如下指令：

mv a on b

将a和b所在列a、b之上的所有积木块恢复到最初其所在列顶部，然后将积木块a搁置在积木块b之上。

mv a ov b

将积木块a之上的所有积木块恢复到最初其所在列顶部，然后将积木块a放置在积木块b所在列的顶端。

st a on b

将积木块b之上的所有积木块恢复到最初其所在列顶部，然后将积木块a及其上的所有积木放置在积木块b之上。

st a ov b

将积木块a及其上的所有积木放置在积木块b所在列的顶端。

xh a an b

交换a和b所在的列。

q

退出

上述命令中，若存在a==b或a、b在同一列上，则为非法指令。机器人会忽略所有的非法指令，不会操作任何的积木块。

输入 输入的第一行为一个整数 n (0 < n < 25) n(0<n<25)，随后有若干行指令，保证所有的指令为上述形式。若指令为q，表示指令结束。

输出 请输出机器人执行所有的指令后的积木状态。记机器人操作之前每个积木所在的列为 i (0 ≤ i < n) i(0≤i<n)，请按样例形式输出每列的所有积木。对任何一列，在一行中输出该列中所有的积木信息。输出格式为：

先输出列号和：，若该列无积木，则直接换行，否则在单个空格后输出各个积木块的编号，编号之间以单个空格分隔。

示例输入

10

mv 9 on 1

mv 8 ov 1

mv 7 ov 1

mv 6 ov 1

st 8 ov 6

st 8 ov 5

mv 2 ov 1

mv 4 ov 9

xh 4 an 9

q

示例输出

0: 0

1: 1 9 2 4

2:

3: 3

4:

5: 5 8 7 6

6:

7:

8:

9:

考查知识点：

1.模拟能力（包括后面电梯II也考查这种能力）；

2.对线性表的操作，主要体现在vector等数组上；

3.对字符串的操作，即对指令的读入与运行。

思路

注意：

我们先看看我们需要关注什么？

题目需要输出操作后每个编号上方放置的积木，所以不难想到我们需要一个能存储序列的及其上方放置的数字，那我们就拿一个二维数组来实现，一个维度存储列号（保持），一个维度存储该列号上的积木号（涉及号码的增添和交换）。

其次，不难发现，多数指令都是在对a，b进行操作，所以对于每个积木的位置信息都极为重要，这样我们才知道该对哪个列号进行查找a，b与进行相应的操作，所以我们需要存储某个积木的所在的列号数。

然后就是对命令的读入，较为简单。

我们就可以采用vector数组来存储积木块

In [ ]:
```
#include<iostream>
#include<vector>
#include<string>
n = 4;
vector<int> piles[25];存储列号与其包含的积木号
int pos[25] //用于记录积木的位置

//初始化,按照上面的例子
for(int i = 0; i < n; i++){
    piles[i].push_back(i);
}
```

补充:

vector等效于c语言的数组，需要#include，故vector pile;就代表创建了一个vector类型的piles文件

而在c语言中 int a[10];代表创建了一个int型的a数组，它有10个元素

在这里，是创建了一个数组piles, 它有n个元素， 每个元素为一个vector类型，相当于一个二维数组

因为vector在对其进行操作时，有很方便的函数可以用，比如push_back就是后加一个数字，所以采用该定义。

一、命令的理解

- **mv a on b**

将a和b所在列a、b之上的所有积木块恢复到最初其所在列顶部，然后将积木块a搁置在积木块b之上。

先找到a，b所在的列号，将其上方的所有积木都放回其积木号对应的列号处，然后在把a放在b所在的编号处（因为b上方已经没有积木了，所以直接放入，不需要考虑插入）

例： a = 3, b = 4;

3: 3 1 2

4: 4 5

mv 3 on 4后

1: 1

2: 2

3: 0

4: 4 3

5: 5

- **mv a ov b**

将积木块a之上的所有积木块恢复到最初其所在列顶部，然后将积木块a放置在积木块b所在列的顶端。

先找到a，将其上方的所有积木都放回其积木号对应的列号处，再把a放在b所在列号的最上方

例： a = 3, b = 4;

3: 3 1 2

4: 4 5

mv 3 ov 4;

1: 1

2: 2

3:

4: 4 5 3

- **st a on b**

将积木块b之上的所有积木块恢复到最初其所在列顶部，然后将积木块a及其上的所有积木放置在积木块b之上。

先找到b的位置，将其上方的所有积木放回其积木号对应的列处，在把a及其上方所有的积木整体移动到b的上方

例： a = 3, b = 4;

3: 3 1 2

4: 4 5

st a on b

3:

4: 4 3 1 2

5: 5

- **st a ov b**

将积木块a及其上的所有积木放置在积木块b所在列的顶端。

这次没有放回了，而是整体放置，将a及其上方整体放在b所在列的上方

例： a = 3, b = 4;

3: 3 1 2

4: 4 5

st a ov b

4: 4 5 3 1 2

- **xh a an b**

交换a和b所在的列。

这个通俗易懂，就是直接交换a，b所在列

例： a = 3, b = 4;

3: 3 1 2

4: 4 5

xh a an b

3: 4 5

4: 3 1 2

以上就是所有命令的理解了！

二、命令的执行

读了这些指令，我们发现其存在许多重复的操作，我们可以将其包装为函数，简化

- 将积木块a之上的所有积木块恢复到最初其所在列顶部

In [ ]:
```
void return_above(int a, int loc,int h, vector<int> piles[],int pos[]) {

    //a代表目标数字，loc代表a所在列号，h代表a在该列的具体位置，稍后我们采用一个函数来查找a的高度

    for (int i = h + 1; i < piles[loc].size(); i++) {
        int b = piles[loc][i];
        piles[b].push_back(b);
        pos[b] = b;
    }

    piles[loc].resize(h + 1);//把piles[loc]的数量调节到0 - h，其后均被删除，达到了清除的目的
}
```

这个函数就完成了该指令，在后续命令的处理时就可以直接调用该函数来解决问题

- **将积木块a及其上的所有积木放置在积木块b所在列的顶端**

将积木块b之上的所有积木块恢复到最初其所在列顶部，然后将积木块a及其上的所有积木放置在积木块b之上可以等同于先将执行一遍return_above,加上上述命令

In [ ]:
```
void move_blocks(int a, int loc, int h, vector<int> piles[], int pos[], int to) {
    //to代表b所在的列号

    for (int i = h; i < piles[loc].size(); i++) {
        int b = piles[loc][i];
```

```
        piles[to].push_back(b);
        pos[b] = to;
    }
    piles[loc].resize(h);
}
```

- **位置的查找**

因为我们始终需要对象积木（a， b等）的高度，所以这个操作也相当频繁，所以也来定义一个函数find_h来查找对象积木的告诉。

In [ ]:
```
int find_h(int a, int loc, vector<int> piles[]) {
    int h;
    for (int i = 0; i < piles[loc].size(); i++) {
        if (piles[loc][i] == a) {
            h = i;
            break;
        }
    }
    return h;
}
```

以上就完成了题干中重复的操作了！

三、从输入到执行

为了方便，我们选择直接输入，判断，然后执行指令

同时，题干中可能存在非法输入，例如a == b 或者 pos[a] == pos[b]。

In [ ]:
```
string cmd;
while (cin >> cmd) {

        if (cmd == "q")break;//退出指令

        int a, b;
        string style;
        cin >> a >> style >> b;

        //输入合法性判断
        if (a == b || pos[a] == pos[b]) continue;

        if (cmd == "mv") {
            if (style == "on") {
                //将a上方积木归位
                int h = find_h(a, pos[a], piles);
                return_above(a, pos[a], h, piles, pos);

                //将b上方积木归位
                h = find_h(b, pos[b], piles);
                return_above(b, pos[b], h, piles, pos);

                //把a放在b的上面
                piles[pos[a]].pop_back();
                piles[pos[b]].push_back(a);
                pos[a] = pos[b];
            }
            else {//"ov"
                //将a上方积木归位
```

```
                int h = find_h(a, pos[a], piles);
                return_above(a, pos[a], h, piles, pos);

                //把a放在b的所在列顶端
                piles[pos[a]].pop_back();
                piles[pos[b]].push_back(a);
                pos[a] = pos[b];
            }
        }
        else if (cmd == "st") {
            if (style == "on") {
                //将b上方积木归位
                int h = find_h(b, pos[b], piles);
                return_above(b, pos[b], h, piles, pos);

                //把a及其上方的所有积木整体移动到b上
                h = find_h(a, pos[a], piles);
                move_blocks(a, pos[a], h, piles, pos, pos[b]);
            }
            else {//"ov"
                int h = find_h(a, pos[a], piles);
                move_blocks(a, pos[a], h, piles, pos, pos[b]);
            }
        }
        else if (cmd == "xh") {
            //记录交换前的位置，用于交换位置号，这可能有点难理解
            int loca = pos[a], locb = pos[b];

            //利用标准库里面swap函数，可以直接交换两者的内容
            swap(piles[loca], piles[locb]);

            //交换积木后，其对应的列号也要整体交换
            for (int x : piles[loca])pos[x] = loca; //piles[loca]中的数的位置全是locb，所以需要
            for (int x : piles[locb])pos[x] = locb;//piles[locb]中的数的位置全是loca，所以需要
        }
    }
```

以上就完成了命令的执行

输入选择二

下面介绍利用来控制输入流的方式，目的是提供一个别的选择，因为这个方法很常用，就当混个脸熟吧！！

In [ ]:
```
#include<sstream>
string line;
while(getline(cin, line)){
    sstringstream ss(line);
    int a, b;
    string cmd, style;
    while(ss >> cmd >> a >> style >> b){
        //以下如上
    }
```

三、输出

按照题目要求即可

完整代码

In [ ]:

```
#include<iostream>
#include<string>
#include<vector>
using namespace std;

void return_above(int a, int loc,int h, vector<int> piles[],int pos[]) {

    //a代表目标数字，loc代表a所在列号，h代表a在该列的具体位置，稍后我们采用一个函数来查找a的高度

    for (int i = h + 1; i < piles[loc].size(); i++) {
        int b = piles[loc][i];
        piles[b].push_back(b);
        pos[b] = b;
    }

    piles[loc].resize(h + 1);//把piles[loc]的数量调节到0 - h，其后均被删除，达到了清除的目的
}

void move_blocks(int a, int loc, int h, vector<int> piles[], int pos[], int to) {
    //to代表b所在的列号

    for (int i = h; i < piles[loc].size(); i++) {
        int b = piles[loc][i];
        piles[to].push_back(b);
        pos[b] = to;
    }
    piles[loc].resize(h);
}

int find_h(int a, int loc, vector<int> piles[]) {
    int h;
    for (int i = 0; i < piles[loc].size(); i++) {
        if (piles[loc][i] == a) {
            h = i;
            break;
        }
    }
    return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(0);cout.tie(0);

    int n;
    cin >> n;
    vector<int> piles[25];
    int pos[25];
    //初始化
    for (int i = 0; i < n; i++) {
        piles[i].push_back(i);
        pos[i] = i;
    }

    string cmd;
    while (cin >> cmd) {

        if (cmd == "q")break;//退出指令

        int a, b;
        string style;
```

```
        cin >> a >> style >> b;

        //输入合法性判断
        if (a == b || pos[a] == pos[b]) continue;

        if (cmd == "mv") {
            if (style == "on") {
                //将a上方积木归位
                int h = find_h(a, pos[a], piles);
                return_above(a, pos[a], h, piles, pos);

                //将b上方积木归位
                h = find_h(b, pos[b], piles);
                return_above(b, pos[b], h, piles, pos);

                //把a放在b的上面
                piles[pos[a]].pop_back();
                piles[pos[b]].push_back(a);
                pos[a] = pos[b];
            }
            else {//"ov"
                //将a上方积木归位
                int h = find_h(a, pos[a], piles);
                return_above(a, pos[a], h, piles, pos);

                //把a放在b的所在列顶端
                piles[pos[a]].pop_back();
                piles[pos[b]].push_back(a);
                pos[a] = pos[b];
            }
        }
        else if (cmd == "st") {
            if (style == "on") {
                //将b上方积木归位
                int h = find_h(b, pos[b], piles);
                return_above(b, pos[b], h, piles, pos);

                //把a及其上方的所有积木整体移动到b上
                h = find_h(a, pos[a], piles);
                move_blocks(a, pos[a], h, piles, pos, pos[b]);
            }
            else {//"ov"
                int h = find_h(a, pos[a], piles);
                move_blocks(a, pos[a], h, piles, pos, pos[b]);
            }
        }
        else if (cmd == "xh") {
            //记录交换前的位置，用于交换位置号，这可能有点难理解
            int loca = pos[a], locb = pos[b];

            //利用标准库里面swap函数，可以直接交换两者的内容
            swap(piles[loca], piles[locb]);

            //交换积木后，其对应的列号也要整体交换
            for (int x : piles[loca])pos[x] = loca; //piles[loca]中的数的位置全是locb，所以需要
            for (int x : piles[locb])pos[x] = locb;//piles[locb]中的数的位置全是loca，所以需要
        }
    }

    for (int i = 0; i < n; ++i) {
        cout << i << ":";
        for (int x : piles[i]) cout << " " << x;
        cout << "\n";
    }
```

```
    return 0;
}
```

声明

本文件处于作者(天佑2502班 郑金理）的自愿制作，可供大家转发分享，作者本人觉得能帮到大家就很开心，同时，如果该文章有任何问题，欢迎联系我，我希望大家能够多多交流，共同成长！可以发邮箱联系我25331053@bjtu.edu.cn 也可以加我微信ZJL3240854272